

Parallelizzazione con MPI del solutore Navier–Stokes–Brinkman

Relazione sulle fasi A e B

1. Di cosa si tratta

Il programma risolve le equazioni di Navier–Stokes–Brinkman su una griglia tridimensionale, con lo schema a separazione delle direzioni visto a lezione. Finora girava su un solo processore.

L’obiettivo è farlo girare su più processori contemporaneamente usando MPI, la libreria standard del calcolo parallelo. L’idea di fondo è semplice: dividere la griglia in blocchi, assegnarne uno a ciascun processore, e far sì che quando a un processore serve un dato che sta a casa del vicino, glielo chieda con un messaggio.

Il lavoro è diviso in fasi. Le fasi A e B, descritte qui, **preparano il terreno**: non cambiano ancora il modo in cui il solutore calcola, ma costruiscono e verificano uno alla volta tutti i pezzi che serviranno.

In questa relazione ogni pezzo è mostrato con il codice vero e spiegato subito dopo.

2. Perché non basta “accendere” MPI

Ci sono due ostacoli, di natura molto diversa.

Primo ostacolo: il codice credeva di avere sempre tutta la griglia

Le dimensioni della griglia erano tre costanti fissate al momento della compilazione: `WIDTH`, `HEIGHT`, `DEPTH`. Ogni ciclo, ogni calcolo di posizione e ogni controllo del tipo “sono sul bordo?” le usava direttamente.

Dividendo la griglia, ciascun processore ne possiede solo una fetta. Il codice non aveva alcun modo di distinguere *la griglia intera* da *la mia parte*: per lui erano la stessa cosa.

Secondo ostacolo: l’algoritmo di Thomas è una catena

Il solutore risolve sistemi tridiagonali con l’algoritmo di Thomas. Il valore in un punto si calcola da quello del punto precedente, che si calcola dal precedente ancora: è una catena rigidamente sequenziale.

Se spezziamo una linea fra due processori, il secondo non può cominciare finché il primo non ha finito. Non avremmo parallelizzato niente: stesso tempo di prima, più il costo dei messaggi. La soluzione, vista a lezione, è il *complemento di Schur*.

3. Fase A — La griglia diventa “un pezzo”

La struttura che sostituisce le costanti

Prima: Tre costanti di compilazione, usate ovunque.

Dopo: Una struttura dati, passata come parametro a ogni funzione che percorre la griglia.

```
typedef struct Decomp {
    int n_global[3]; /* dimensione della griglia intera */
    int n[3]; /* celle che possiedo io */
    int start[3]; /* indice globale della mia prima cella */
    size_t stride[3]; /* distanza in memoria fra celle vicine */
    size_t base; /* offset della prima cella posseduta */
    int is_first[3]; /* tocco la parete inferiore? */
}
```

```

    int is_last[3]; /* tocco la parete superiore? */
    size_t n_cells; /* celle da allocare per un campo */
} Decomp;

```

Con una griglia $32 \times 16 \times 24$ su un solo processore i campi valgono: `n_global` e `n` entrambi $\{32, 16, 24\}$, `start` $\{0, 0, 0\}$, `stride` $\{1, 32, 512\}$, `base` 0, tutti i flag a 1, e `n_cells` = 12 288.

Il campo che porta tutto il peso è `stride`. Significa: per spostarsi di una cella lungo *Y* bisogna avanzare di 32 posizioni nell’array, e per cambiare piano di 512 (cioè 32×16). Averlo memorizzato come *dato* invece di ricostruirlo ogni volta dalle dimensioni è ciò che renderà indolore la fase C.

Come si raggiunge una cella

```

static inline size_t decomp_index(const Decomp *d, int i, int j, int k) {
    return d->base + (size_t)i * d->stride[0] + (size_t)j * d->stride[1] +
           (size_t)k * d->stride[2];
}

static inline int decomp_global(const Decomp *d, int i, int component) {
    return d->start[component] + i;
}

```

La prima funzione traduce una posizione nella griglia in una posizione in memoria. Con i valori dell’esempio dà $0 + i + 32*j + 512*k$, cioè esattamente il calcolo che il codice faceva prima a mano: stessa aritmetica, ma con i numeri presi dalla struttura.

La seconda traduce un indice *locale* (“la mia terza cella”) in un indice *globale* (“la cella numero 15 del dominio”). Serve ogni volta che si calcola una coordinata fisica o si chiede se siamo su una parete.

Come si riempie la struttura

```

static void decomp_set_layout(Decomp *d, int halo) {
    size_t allocated_x = (size_t)d->n[0] + 2 * (size_t)halo;
    size_t allocated_y = (size_t)d->n[1] + 2 * (size_t)halo;
    size_t allocated_z = (size_t)d->n[2] + 2 * (size_t)halo;

    d->stride[0] = 1;
    d->stride[1] = allocated_x;
    d->stride[2] = allocated_x * allocated_y;
    d->base = (size_t)halo * (d->stride[0] + d->stride[1] + d->stride[2]);
    d->n_cells = allocated_x * allocated_y * allocated_z;
}

```

Questa è l’unica funzione che decide dove stanno fisicamente i dati. Il parametro `halo` è il margine di celle lasciato su ogni faccia: oggi vale zero, in fase C diventerà uno.

Il punto importante: **aggiungere il margine cambia solo questa funzione**. Con `halo` = 1 gli stride diventano $\{1, 34, 612\}$ e `base` diventa 647 invece di 0 – e le centinaia di chiamate a `decomp_index` sparse nel codice continuano a funzionare senza essere toccate.

I quattro tipi di modifica

Ogni singola modifica della fase A rientra in uno di questi quattro casi. I primi tre compaiono tutti insieme in questo ciclo, che riempie un campo di velocità.

Prima

```

size_t off = 0;
for (int k = 0; k < DEPTH; k++) {
    for (int j = 0; j < HEIGHT; j++) {
        for (int i = 0; i < WIDTH; i++) {

```

```

    Real x = centered_physical_coord(i, 0);
    ...
    v_x[off] = vector_fn(staggered_physical_coord(i, 0), y, z, time, 0);
    off++;
  }
}
}

```

Dopo

```

for (int k = 0; k < d->n[2]; k++) {
  int gk = decomp_global(d, k, 2);
  for (int j = 0; j < d->n[1]; j++) {
    int gj = decomp_global(d, j, 1);
    size_t row = decomp_index(d, 0, j, k);
    for (int i = 0; i < d->n[0]; i++) {
      int gi = decomp_global(d, i, 0);
      Real x = centered_physical_coord(gi, 0);
      ...
      size_t off = row + (size_t)i;
      v_x[off] = vector_fn(staggered_physical_coord(gi, 0), y, z, time, 0);
    }
  }
}
}

```

Tre cose sono cambiate.

- **Gli estremi dei cicli.** Da “scorro tutta la griglia” a “scorro le celle che possiedo”.
- **La posizione in memoria.** Il contatore `off++` avanzava di una posizione alla volta: funziona solo se le celle sono perfettamente consecutive. Con il margine non lo saranno più – dopo l’ultima cella di una riga vengono le celle di contorno, poi la riga successiva – e `off++` finirebbe dentro il margine. Ora la posizione si calcola.
- **La coordinata fisica.** Da `i` a `decomp_global(d, i, 0)`. La posizione di una cella nel mondo fisico dipende da dove sta nel *dominio*, non da dove sta nel mio pezzo di memoria. Senza questa conversione, il secondo processore calcolerebbe le forze e le condizioni al contorno come se si trovasse all’origine del dominio.

Il quarto caso è il più insidioso, e riguarda i controlli sul bordo.

Prima

```

if (i < 1 || i >= WIDTH - 1 || j < 1 || j >= HEIGHT || ...) {
  return 0.0;
}

```

Dopo

```

int gi = decomp_global(d, i, 0);
int gj = decomp_global(d, j, 1);
if (gi < 1 || gi >= d->n_global[0] - 1 || gj < 1 || ...) {
  return 0.0;
}

```

Prima, “ultima cella del mio array” e “sono sulla parete fisica” erano la stessa condizione. Con più processori le due cose si separano: un processore in mezzo al dominio ha una sua ultima cella, ma lì non c’è nessuna parete, c’è il vicino.

Sbagliare questo controllo crea *pareti fantasma* dentro il fluido: la simulazione gira, i numeri sembrano plausibili, e sono sbagliati. È il tipo di errore che si scopre solo confrontando i risultati fra numeri di processori diversi.

4. Fase B — I mattoni del parallelo

La fase B è divisa in quattro passi. Due dei quattro non usano MPI affatto: è voluto, e serve a separare la domanda “la matematica è giusta?” dalla domanda “la comunicazione è giusta?”.

B0 — Un muro fra MPI e il solutore

Prima: Nessun supporto per MPI.

Dopo: Due file nuovi che sono l’unico posto del progetto a conoscere MPI.

```
/* parallel.h */
void par_init(int *argc, char ***argv);
void par_finalize(void);
int par_rank(void); /* chi sono io: da 0 a par_size()-1 */
int par_size(void); /* quanti siamo */

/* parallel.c */
#ifdef USE_MPI
#include <mpi.h>
int par_rank(void) {
    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    return rank;
}
#else
int par_rank(void) { return 0; } /* siamo un processo solo */
#endif
```

Il solutore chiama `par_rank()` e non sa che sotto ci sia `MPI_Comm_rank`. Non lo sa e non deve saperlo.

Il vantaggio non è estetico. Senza l’opzione di compilazione, le funzioni diventano banali e il programma seriale continua a funzionare come prima, senza MPI installato. La versione seriale resta così un **termine di paragone**: ogni volta che il parallelo darà un numero sospetto, lo confronteremo con la versione che sappiamo essere giusta.

B1 — Spezzare la catena, senza ancora usare MPI

Prima il metodo sequenziale, che resta il riferimento.

```
void thomas_solve(int n, const Real *a, const Real *b, const Real *c,
                  const Real *f, Real *x, Real *scratch) {
    Real denominator = b[0];
    scratch[0] = c[0] / denominator;
    x[0] = f[0] / denominator;

    for (int i = 1; i < n; i++) { /* eliminazione */
        denominator = b[i] - a[i] * scratch[i - 1];
        scratch[i] = c[i] / denominator;
        x[i] = (f[i] - a[i] * x[i - 1]) / denominator;
    }

    for (int i = n - 2; i >= 0; i--) { /* sostituzione */
        x[i] -= scratch[i] * x[i + 1];
    }
}
```

Si vede bene la catena: nel primo ciclo il valore in `i` dipende da quello in `i-1`. Non c’è modo di calcolarli in parallelo.

Il complemento di Schur rompe la catena così: dentro il mio blocco, la soluzione è la somma di due contributi. Il primo è la risposta al carico che ho io, e lo calcolo subito da solo. Il secondo

è la risposta ai valori alle mie due estremità, che ancora non conosco – ma posso calcolare quanto “pesa” ciascuna estremità sul mio interno, con due sole risoluzioni locali in più.

```
/* ---- 1. ogni blocco per conto suo: tre risoluzioni locali ---- */
thomas_solve(len, a+begin, b+begin, c+begin, f+begin, y+begin, scratch);

if (p > 0) { /* influenza dell'estremità sinistra */
    memset(rhs, 0, len * sizeof(Real));
    rhs[0] = a[begin];
    thomas_solve(len, a+begin, b+begin, c+begin, rhs, lft+begin, scratch);
}

if (has_right) { /* influenza dell'estremità destra */
    memset(rhs, 0, len * sizeof(Real));
    rhs[len - 1] = c[internal_end - 1];
    thomas_solve(len, a+begin, b+begin, c+begin, rhs, rgt+begin, scratch);
}
```

y è la risposta al carico proprio; lft e rgt sono le due funzioni di influenza, ottenute risolvendo lo stesso sistema con un termine noto che vale zero ovunque tranne che a un'estremità. Nessuna di queste tre risoluzioni guarda i dati di un altro blocco.

A quel punto le uniche incognite rimaste sono i punti di giunzione fra blocchi. Sostituendo l'espressione della soluzione interna nelle equazioni scritte in quei punti si ottiene un sistema nelle sole giunzioni, che risulta essere di nuovo tridiagonale:

```
/* ---- 2. il sistema rimasto sulle giunzioni ---- */
int m = end - 1; /* il punto di giunzione */
int left = m - 1; /* ultimo punto interno del blocco p */
int right = next_begin; /* primo punto interno del blocco p+1 */

ra[p] = -a[m] * lft[left];
rb[p] = b[m] - a[m] * rgt[left] - c[m] * lft[right];
rc[p] = -c[m] * rgt[right];
rf[p] = f[m] - a[m] * y[left] - c[m] * y[right];

thomas_solve(interfaces, ra, rb, rc, rf, rx, scratch);
```

Con quattro blocchi questo sistema ha tre incognite, contro le centoventotto di partenza: è minuscolo. Risolto quello, ogni blocco ricompone la sua parte:

```
/* ---- 3. ogni blocco di nuovo per conto suo ---- */
Real x_left = (p > 0) ? rx[p - 1] : (Real)0;
Real x_right = has_right ? rx[p] : (Real)0;

for (int i = begin; i < internal_end; i++) {
    x[i] = y[i] - x_left * lft[i] - x_right * rgt[i];
}
```

Questa riga è la sovrapposizione degli effetti scritta in codice: la risposta al mio carico, meno il contributo di ciascuna estremità pesato per il valore che quell'estremità ha effettivamente assunto.

Il punto importante: **non è un'approssimazione**. È lo stesso identico risultato del metodo sequenziale, con le operazioni fatte in un ordine diverso.

In questo passo i “blocchi” sono ancora fette di un unico array dentro un unico processo, e non si comunica niente. Serve a verificare la matematica quando è ancora ispezionabile con un debugger ordinario.

B2 — Dare a ciascun processore la sua fetta

Prima: I processori esistevano ma nessuno diceva loro quale parte della griglia gli spettasse.

Dopo: I processori sono disposti su una griglia tridimensionale, e ognuno ne ricava la propria fetta.

```
void par_topology_init(const int procs[3]) {
    int dims[3] = {procs[0], procs[1], procs[2]};
    int periods[3] = {0, 0, 0};

    MPI_Dims_create(par_size(), 3, dims);
    MPI_Cart_create(MPI_COMM_WORLD, 3, dims, periods, 0, &cart_comm);
    MPI_Cart_get(cart_comm, 3, cart_dims, periods, cart_coords);

    for (int c = 0; c < 3; c++) { /* un gruppo per direzione */
        int keep[3] = {0, 0, 0};
        keep[c] = 1;
        MPI_Cart_sub(cart_comm, keep, &line_comm[c]);
    }
}
```

`MPI_Dims_create` riempie solo le direzioni lasciate a zero: passando $\{1, 1, 0\}$ si ottiene una divisione a fette lungo Z , passando $\{0, 0, 0\}$ è MPI a scegliere una forma equilibrata (con otto processori, $2 \times 2 \times 2$).

L'ultimo ciclo crea tre gruppi di processori, uno per direzione: sono i gruppi che si scambieranno le giunzioni nel complemento di Schur.

```
void decomp_init_mpi(Decomp *d) {
    int dims[3], coords[3];
    par_dims(dims);
    par_coords(coords);

    for (int c = 0; c < 3; c++) {
        int begin, end;
        decomp_share(d->n_global[c], dims[c], coords[c], &begin, &end);

        d->n[c] = end - begin;
        d->start[c] = begin;
        d->is_first[c] = (coords[c] == 0);
        d->is_last[c] = (coords[c] == dims[c] - 1);
    }
    decomp_set_layout(d, 0);
}
```

Qui si vede il senso della fase A: è *l'unica* funzione che cambia rispetto alla versione seriale. `n` diventa più piccolo, `start` non è più zero, e i flag delle pareti diventano veri solo per i processori che stanno davvero agli estremi. Tutto il resto del solutore non se ne accorge.

Con 24 celle lungo Z e cinque processori i blocchi escono di dimensione 5, 5, 5, 5, 4: il resto della divisione viene distribuito una cella per volta ai primi blocchi, così le dimensioni differiscono al massimo di uno.

B3 — Il complemento di Schur su processori veri

Prima: Il complemento di Schur funzionava su fette di un array, in un solo processo.

Dopo: Funziona con un blocco per processore, e i valori che prima si leggevano dall'array del vicino ora viaggiano come messaggi.

Le fasi 1 e 3 restano identiche a prima e non comunicano. Cambia solo la fase 2:

```

/* Il mio vicino di sinistra ha bisogno di sapere quanto vale
   il mio primo punto interno nelle tre risposte locali. */
Real mine[3] = {y[0], lft[0], rgt[0]};
Real from_right[3] = {0, 0, 0};
par_shift_real(axis, -1, mine, from_right, 3);

/* La riga del sistema sulle giunzioni che spetta a me.
   L'ultimo processore non ha giunzioni e manda una riga innocua. */
Real row[4] = {0, 1, 0, 0};

if (has_right) {
    int m = n_local - 1;
    int left = len - 1;
    row[0] = -a[m] * lft[left];
    row[1] = b[m] - a[m] * rgt[left] - c[m] * from_right[1];
    row[2] = -c[m] * from_right[2];
    row[3] = f[m] - a[m] * y[left] - c[m] * from_right[0];
}

/* Tutti ricevono tutte le righe, e tutti risolvono il sistemino:
   cosi' nessuno deve rispedire indietro la soluzione. */
par_line_allgather(axis, row, 4, rows);

```

Confrontando con il codice di B1 si vede che le formule sono le stesse: dove prima c'era `lft[right]` – un valore letto dall'array del blocco vicino – ora c'è `from_right[1]`, cioè lo stesso valore arrivato per posta.

Lo scambio col vicino è implementato così:

```

void par_shift_real(int axis, int step,
                   const Real *send, Real *recv, int count) {
    MPI_Sendrecv(send, count, PAR_REAL, mpi_neighbor(axis, step), 0,
                 recv, count, PAR_REAL, mpi_neighbor(axis, -step), 0,
                 cart_comm, MPI_STATUS_IGNORE);
}

```

Due dettagli meritano una nota.

`MPI_Sendrecv` invece di una `Send` seguita da una `Recv`: due processori affacciati devono scambiarsi dati nello stesso istante, e con chiamate separate si bloccherebbero a vicenda non appena i messaggi superano la soglia oltre la quale MPI smette di conservarli in un'area temporanea.

`mpi_neighbor` restituisce un valore speciale, `MPI_PROC_NULL`, dove il vicino non c'è. MPI tratta un invio o una ricezione verso il vuoto come un'operazione che non fa niente, quindi non serve nessun caso particolare per i processori sul bordo.

Il conto totale della comunicazione è sorprendentemente piccolo:

Fase	Comunicazione
1. Calcolo locale (tre risoluzioni)	nessuna
2. Sistema sulle giunzioni	3 numeri col vicino, 4 per processore
3. Ricomposizione locale	nessuna

Tutto il resto è calcolo che ciascun processore fa per conto proprio: è esattamente ciò che serve perché il parallelo convenga.

5. Come abbiamo verificato

Ogni passo ha il suo controllo automatico, che risponde “passato” o “fallito” invece di stampare numeri da interpretare a occhio.

Controllo	Che cosa verifica	Esito
Risultati invariati	Il solutore dà gli stessi numeri di prima della fase A	identici, cifra per cifra
Disposizione in memoria	Cambiando come i dati stanno in RAM, il risultato non cambia	zero celle diverse
Schur (locale)	Stessa risposta del metodo sequenziale, 24 configurazioni	scarto $\sim 10^{-17}$
Fette dei processori	I blocchi coprono la griglia una volta sola	passato da 1 a 8 processori
Schur (distribuito)	Stessa risposta del sequenziale, con i messaggi	scarto $\sim 10^{-17}$

Lo scarto di 10^{-17} è la precisione della macchina: non è “abbastanza vicino”, è la stessa soluzione con le somme fatte in ordine diverso. Con un solo processore lo scarto è esattamente zero, perché il metodo degenera in Thomas puro.

Il controllo che le fette combacino

Merita una menzione perché è tutto in un numero solo:

```
static long long owned_index_sum(const Decom *d) {
    long long total = 0;
    for (int k = 0; k < d->n[2]; k++)
        for (int j = 0; j < d->n[1]; j++)
            for (int i = 0; i < d->n[0]; i++)
                total += (long long)decomp_global(d, k, 2) * plane
                    + (long long)decomp_global(d, j, 1) * d->n_global[0]
                    + decomp_global(d, i, 0);
    return total;
}
```

Ogni processore somma l’“etichetta” (l’indice globale) delle celle che possiede, e il totale su tutti deve valere la somma di tutte le etichette della griglia, che si conosce in forma chiusa. Un buco lo abbassa, una sovrapposizione lo alza, un blocco spostato lo cambia comunque: tre difetti diversi, una sola verifica.

Abbiamo verificato anche i controlli stessi

Un controllo che passa sempre non dimostra niente. Per esserne certi abbiamo introdotto di proposito degli errori nel codice, uno alla volta, e verificato che venissero scoperti. Su ventiquattro errori introdotti, alla fine tutti sono stati scoperti.

Questa verifica ha ripagato subito, perché ha rivelato un difetto *nei controlli stessi*. Il calcolo dello scarto massimo era scritto così:

```
if (difference > worst) {
    worst = difference;
}
```

Rompendo di proposito una comunicazione, il risultato diventava “non numerico” – il valore che si ottiene, ad esempio, dividendo per zero. Ma per quei valori *ogni confronto risulta falso*: `difference > worst` era falso, il valore non entrava mai nel massimo, e il controllo dichiarava “passato” su un risultato completamente rotto.

La correzione è invertire il confronto:

```
if (!(difference <= worst)) {
    worst = difference;
}
```


Sui numeri normali le due scritture sono equivalenti; su un valore non numerico la seconda risulta vera, quindi il valore diventa il massimo e l'errore emerge. Lo stesso difetto era presente anche nella raccolta del massimo fra processori, ed è stato corretto allo stesso modo. Riguardava due controlli su cinque.

6. A che punto siamo

Quello che funziona: tutti i mattoni del parallelo esistono e sono verificati singolarmente. Il programma compila e gira sia in versione seriale sia con MPI, con e senza le ottimizzazioni vettoriali già presenti nel progetto, senza alcun avviso del compilatore.

Quello che ancora non succede: il solutore *non è ancora parallelo*. Lanciandolo oggi su quattro processori, ognuno risolve lo stesso identico problema per intero, senza parlare con gli altri: quattro volte la memoria, nessun guadagno.

Questo è voluto. Collegare i pezzi prima di avere gli scambi al contorno produrrebbe risultati sbagliati, e sarebbe difficile capire dove.

7. Prossimi passi

Fase C — Celle di contorno

Per calcolare vicino al proprio bordo un processore ha bisogno di una fascia sottile di dati del vicino. Si aggiunge un bordo di celle “in prestito” e lo scambio che le riempie. Si scambia circa il 3% dei dati per poter calcolare il restante 97% in autonomia. Dal lato del codice, basterà chiamare `decomp_set_layout` con margine uno.

Fase D — Primo calcolo davvero parallelo

Si collegano i pezzi. Il controllo decisivo: gli stessi errori di soluzione con 1, 2, 4 e 8 processori. Poiché il metodo è esatto, il numero di processori non deve cambiare il risultato.

Fase E — Divisione nelle tre direzioni

Finora la griglia si divide solo lungo una direzione. Si estende alle altre due riusando lo stesso codice, con un asse diverso.

Fase F — Risultati e file di uscita

Alcune quantità, come le norme dell'errore, sono somme su tutta la griglia e vanno raccolte da tutti i processori. E ogni processore deve scrivere la propria parte del file di uscita.

Fase G — Misure di prestazione

I grafici che mostrano il guadagno: quanto si accorcia il tempo aumentando i processori, e quanto cresce il problema affrontabile a parità di tempo per processore.

Le fasi A e B sono completate e verificate. Il lavoro che segue si appoggia su pezzi già collaudati singolarmente: quando qualcosa non funzionerà, sapremo dove non guardare.